Reforzamiento - Fundamentos de Programación

SEMANA 3 · JAVA

Guía de conceptos esenciales — estructuras de selección, estructuras de repetición, variables, acumuladores y entrada/salida de datos.

Docente: Jorge Carmona

Índice de contenidos

¿Qué aprenderás esta semana?

Esta guía cubre los bloques fundamentales que necesitas dominar para resolver problemas de programación con Java. Cada concepto se presenta con ejemplos y mini proyectos para practicar.

01

Variables y tipos de datos

Declaración, tipos básicos y convención camelCase

02

Estructuras de selección

if, else if, else — operadores relacionales y lógicos

03

Estructuras de repetición

Ciclos for y while — cuándo y cómo usarlos

04

Contadores y acumuladores

Variables especiales para sumar y contar dentro de ciclos

05

Entrada y salida: Scanner y printf

Leer datos del usuario y mostrar resultados con formato

06

Integración: if/else dentro de ciclos

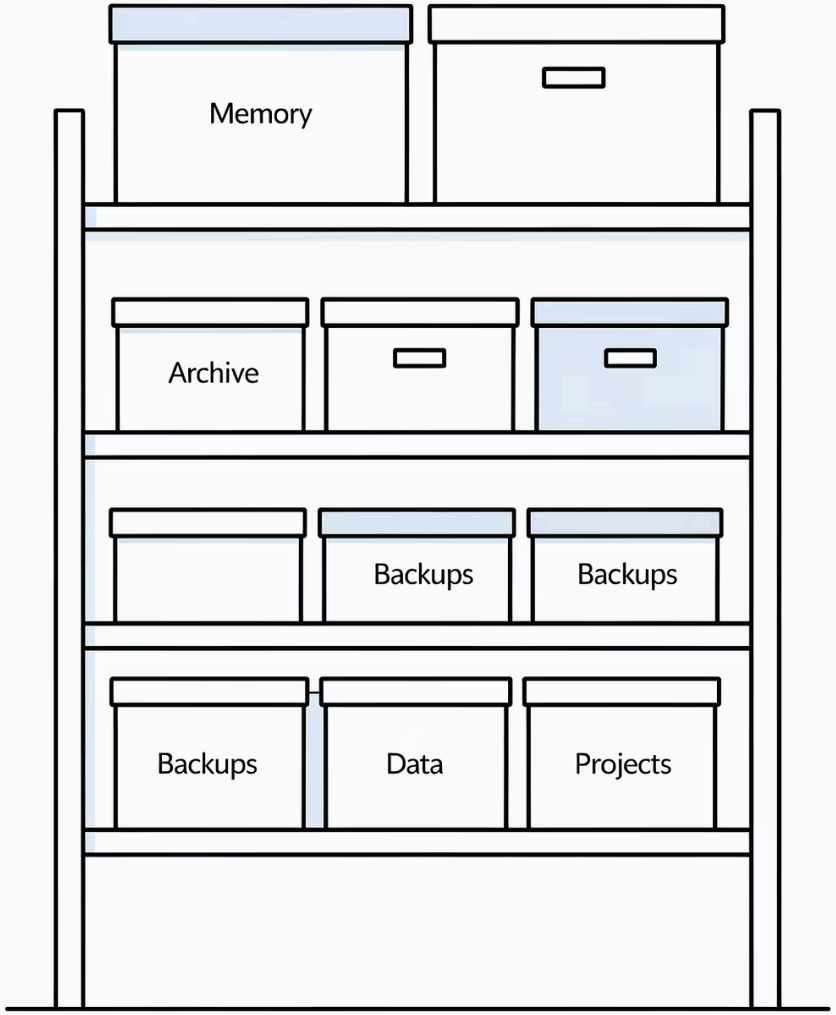
El patrón central de la mayoría de los programas de esta unidad

CONCEPTO 1

Variables, tipos de datos y nombrado

Una **variable** es un espacio en la memoria del computador donde el programa guarda un dato que puede cambiar durante la ejecución. Tiene tres características fundamentales: un **nombre**, un **tipo** y un **valor**.

Analogía: Imagina una caja con etiqueta. La etiqueta es el nombre, el tamaño de la caja determina qué tipo de dato cabe, y lo que metes dentro es el valor.



Tipos de datos más usados en Java

Tipo	¿Qué almacena?	Ejemplo de declaración	Rango / Precisión
`int`	Números enteros	`int edad = 25;`	−2.147.483.648 a 2.147.483.647
`double`	Números con decimales	`double precio = 99.90;`	~15 dígitos de precisión
`boolean`	Verdadero o falso	`boolean activo = true;`	true / false
`String`	Texto (cadena)	`String nombre = "Ana";`	Cualquier texto

Reglas camelCase en Java

- Empieza con minúscula: `edad`, `totalVentas`
- Cada palabra siguiente en mayúscula: `precioUnitario`
- Nombres descriptivos — evita `a`, `x`, `temp`
- Sin acentos ni espacios: no escribas `número` ni `total ventas`

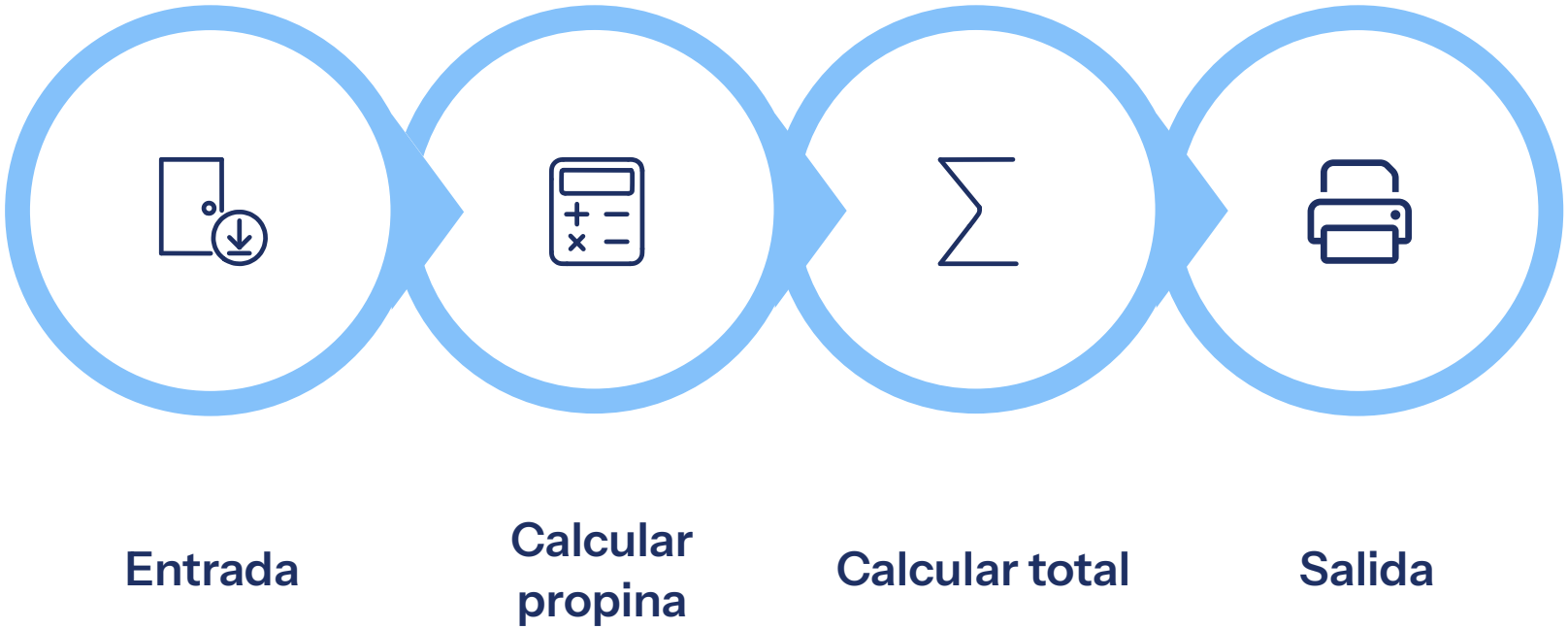
¿Por qué elegir el tipo correcto?

Usar `int` para el porcentaje y `double` para el importe no es arbitrario: `double` maneja decimales con precisión, mientras que `int` descarta la parte fraccionaria. Elegir mal el tipo provoca pérdidas de precisión que son difíciles de depurar.

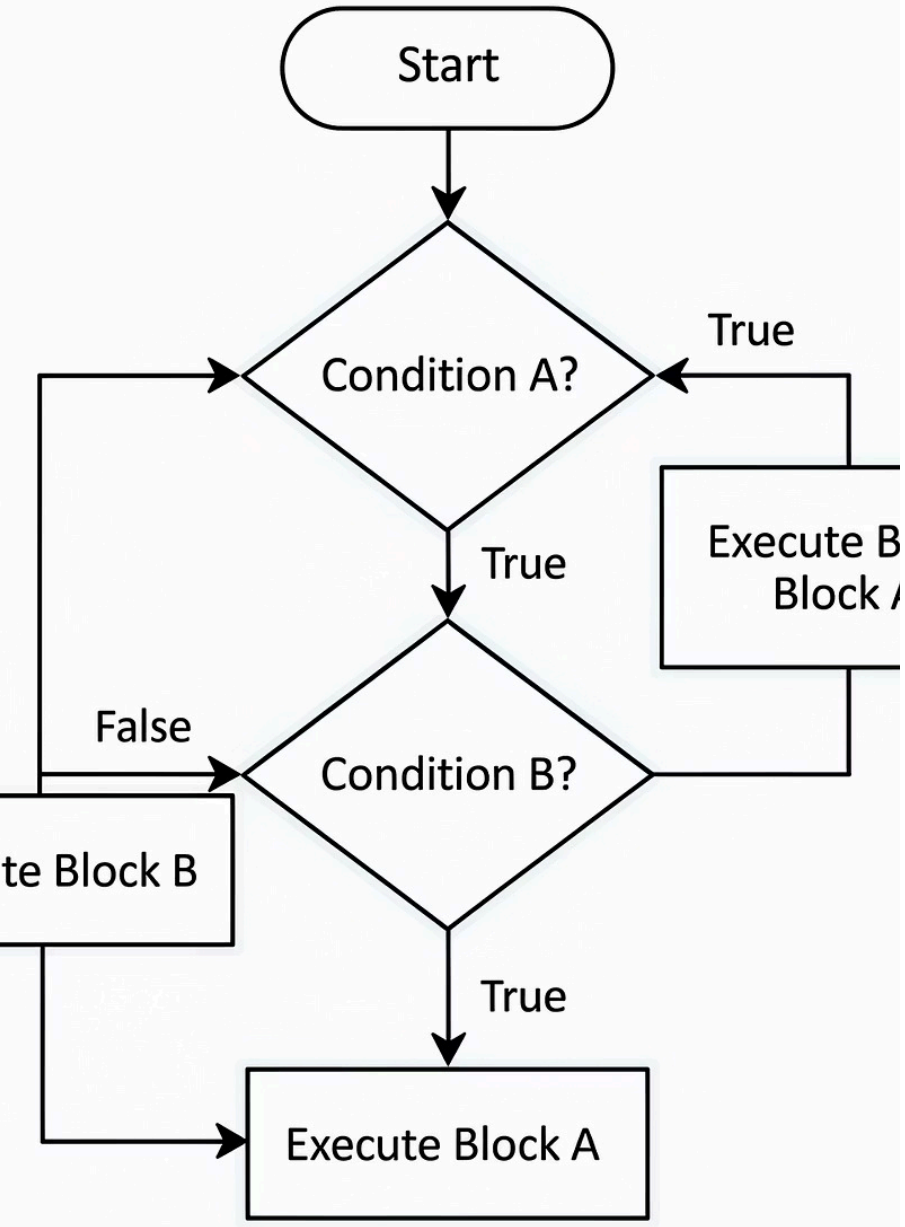
MINI PROYECTO 1

Calculadora de propina

Practica la declaración de variables, las operaciones aritméticas y el uso de `printf` para mostrar resultados con formato. El programa solicita el total de la cuenta y el porcentaje de propina, y calcula el importe y el total a pagar.



Observa que `totalCuenta` y `propina` son `double` porque manejan decimales, mientras que `porcentaje` es `int`. Esta elección de tipos garantiza que el cálculo sea preciso y no pierda decimales.



CONCEPTO 2

Estructuras de selección: if / else if / else

Un programa toma decisiones según las condiciones del momento. La estructura `if/else` permite ejecutar bloques de código distintos dependiendo de si una condición es verdadera o falsa. Sin ella, el programa siempre haría lo mismo sin importar los datos de entrada.

> Una vez que una condición es `true`, Java entra a ese bloque y **salta todos los else if y else** que siguen. No evalúa el resto aunque pudieran ser también verdaderos.

Operadores relacionales y lógicos

Operadores relacionales

Operador	Significado	Ejemplo
`==`	Igual a	`5 == 5 → true`
`!=`	Distinto de	`5 != 3 → true`
`>`	Mayor que	`10 > 7 → true`
`<`	Menor que	`3 < 1 → false`
`>=`	Mayor o igual	`5 >= 5 → true`
`<=`	Menor o igual	`4 <= 3 → false`

Operadores lógicos

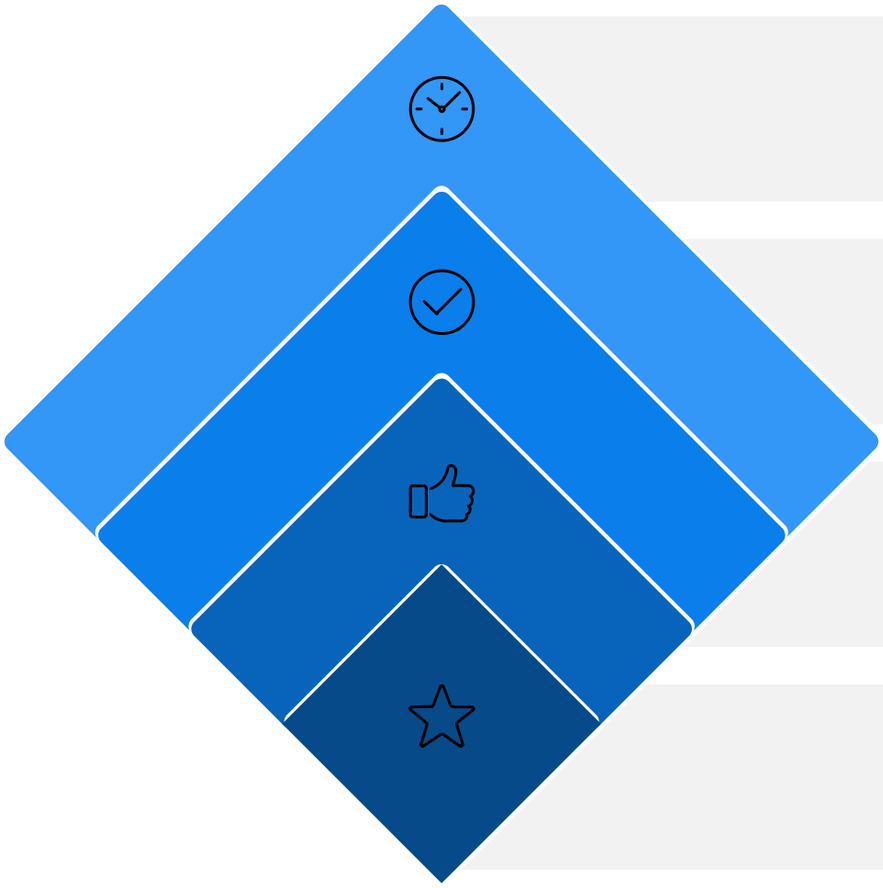
Operador	Significado	Resultado
`&&`	Y — ambas deben ser true	true solo si las dos son true
`||`	O — al menos una true	true si una o ambas son true
`!`	No — invierte el resultado	true si la condición era false

Ejemplo: `edad >= 18 && tieneLicencia` es `true` solo si se cumplen ambas condiciones simultáneamente.

MINI PROYECTO 2

Clasificador de notas

Practica el uso de rangos con `else if` y comprende por qué el **orden de las condiciones importa**. El programa lee una nota entre 1.0 y 7.0 y muestra la categoría correspondiente.



Reprobado
nota >= 1.0

Aprobado
nota >= 4.0

Muy bien
nota >= 5.0

Excelente
nota >= 6.0

⚠️ **Error clásico de orden:** si pones `nota >= 1.0` primero, todas las notas válidas entran ahí y el resto de condiciones nunca se evalúa. Regla práctica: cuando uses rangos numéricos con `else if`, empieza siempre por el extremo más restrictivo (el mayor) y avanza hacia el menor.

MINI PROYECTO 3

Detector de valores borde

Aprende a verificar **casos límite**: los valores exactos en los extremos de un rango son los más propensos a errores. Este proyecto trabaja con una variable `velocidad` y clasifica el estado del vehículo.

velocidad < 0

Velocidad inválida — valor imposible en la realidad

velocidad == 0

Vehículo detenido — caso límite inferior

velocidad <= 60

Zona urbana — velocidad permitida

velocidad <= 100

Carretera — velocidad permitida

else

Exceso de velocidad — más de 100 km/h

Prueba mentalmente con: `−5, 0, 1, 60, 61, 100, 101, 150`. Verifica que cada valor entre al caso correcto antes de escribir una sola línea de código.

CONCEPTO 3

Estructuras de repetición: for y while

Un ciclo (*loop*) permite ejecutar un bloque de código varias veces sin copiar y pegar. Siempre que pienses *"esto hay que hacerlo N veces"* o *"esto hay que hacerlo mientras se cumpla algo"*, necesitas un ciclo.

Ciclo for

Úsalo cuando **sabes exactamente cuántas veces** necesitas iterar. El contador se define en la cabecera: inicialización, condición y actualización en una sola línea.

Ciclo while

Úsalo cuando **la cantidad de iteraciones depende de una condición** que puede cambiar. El contador debe declararse antes y actualizarse dentro del cuerpo del ciclo.

Si la condición del `while` nunca se vuelve `false`, el programa entra en un **bucle infinito** y se congela. Siempre verifica que algo dentro del `while` modifique la condición.

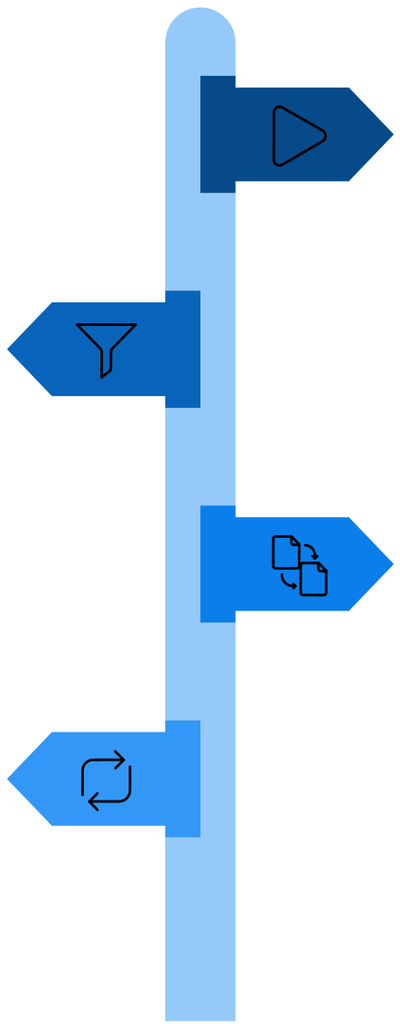
Comparación: for vs while

for — cuando sabes cuántas veces

- Contador definido en la cabecera
- Riesgo: error de límite (off-by-one)
- Ejemplo típico: iterar N botellas, tabla de multiplicar

while — cuando depende de una condición

- Contador declarado antes, actualizado dentro
- Riesgo: bucle infinito si la condición nunca cambia
- Ejemplo típico: pedir datos hasta que sean válidos



For: Inicialización

Contador definido en la cabecera.

For: Condición

Comprueba límite; riesgo off-by-one.

For: Actualización

Incremento/ajuste después de cada iteración.

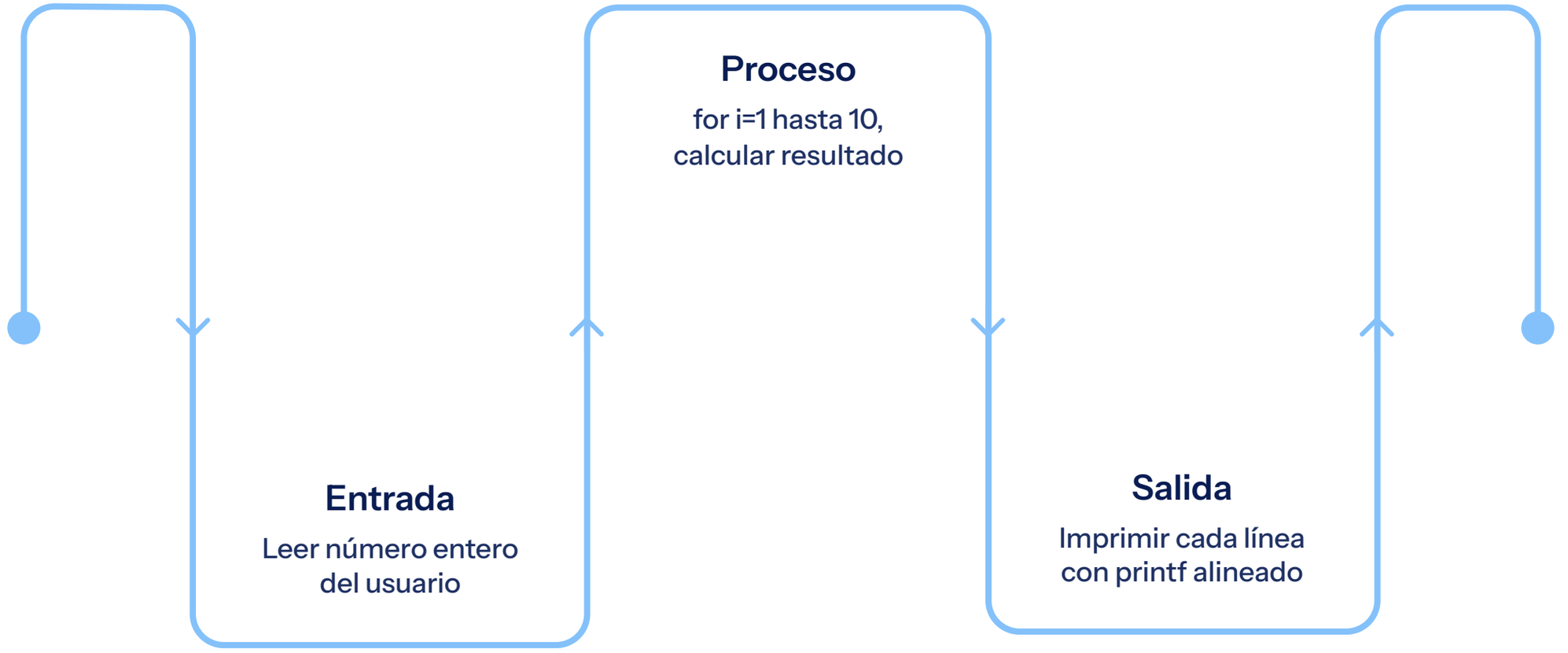
While: Condición

Dependiente de estado; contador antes.

MINI PROYECTO 4

Tabla de multiplicar

Comprende la estructura básica del `for` con un contador y visualiza las iteraciones. El programa solicita un número y muestra su tabla de multiplicar del 1 al 10 con formato alineado usando `printf`.

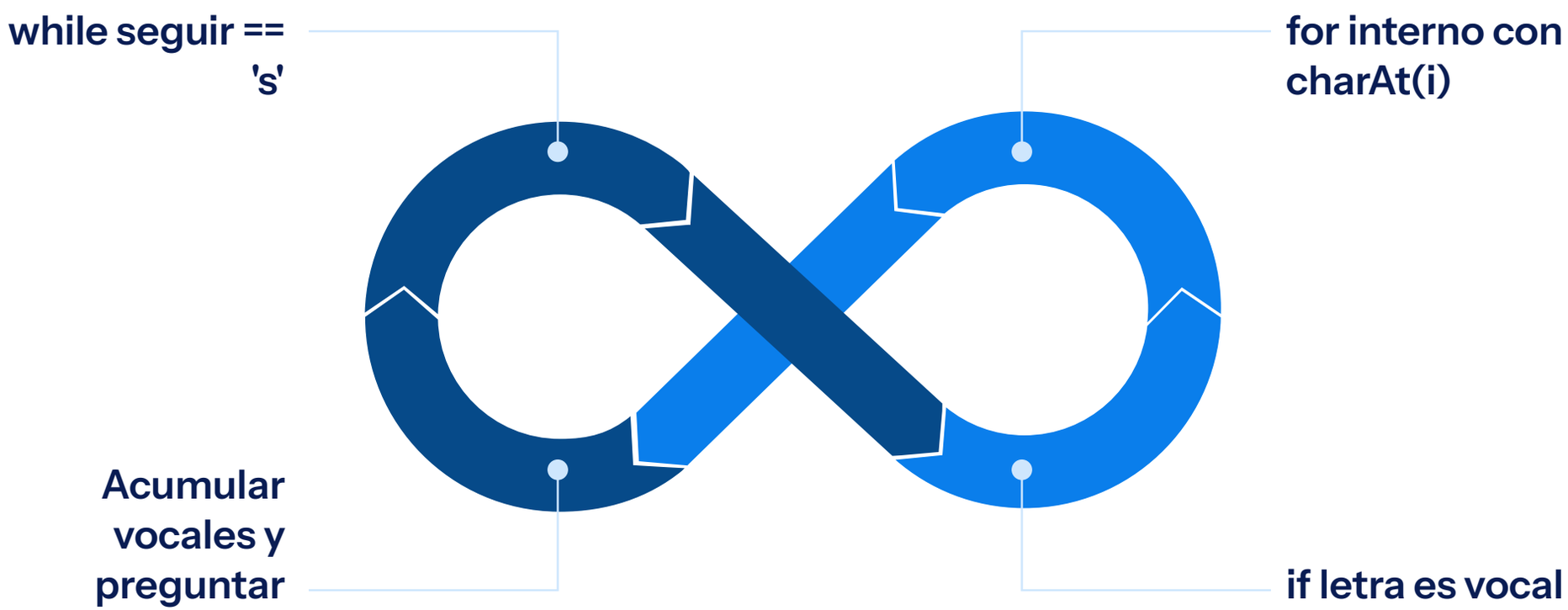


💡 El especificador `%2d` en `printf` reserva 2 espacios para el número, garantizando que todas las líneas queden alineadas visualmente. Esta es una buena práctica para presentar tablas en consola.

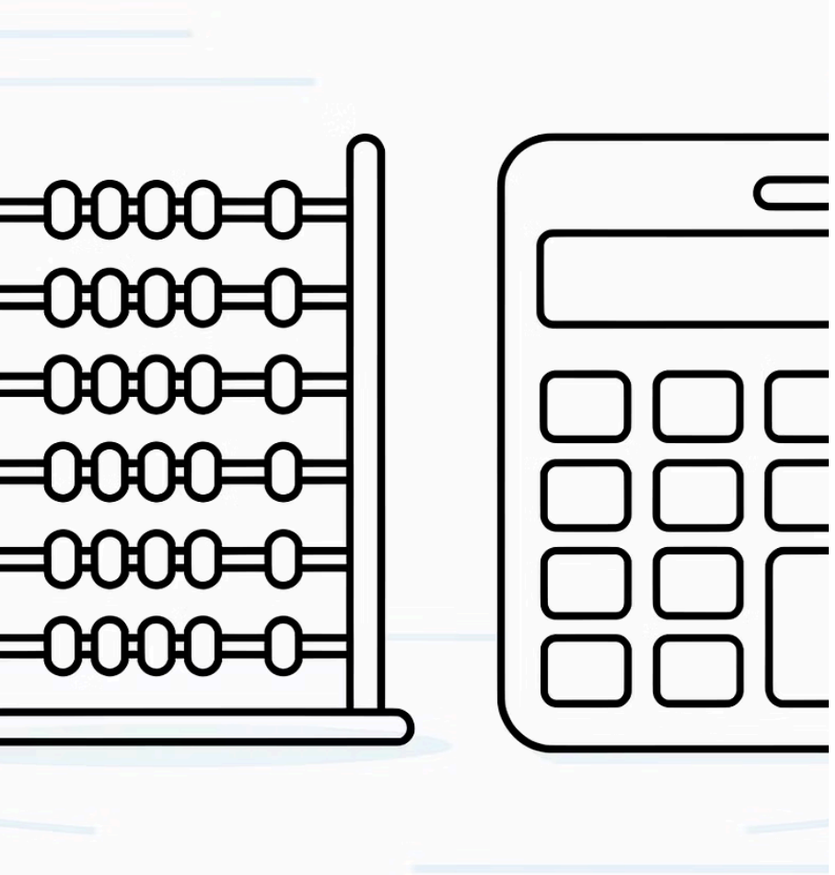
MINI PROYECTO 5

Contador de vocales

Usa un `while` con condición de entrada controlada por el usuario. El programa procesa palabras una a una y acumula el total de vocales encontradas, hasta que el usuario indica que desea salir.



Este mini proyecto muestra un **ciclo for dentro de un while** — lo que se llama **ciclos anidados**. Es perfectamente válido: el `for` interno recorre cada letra de la palabra, y el `while` externo controla cuántas palabras se procesan.



CONCEPTO 4

Variables especiales: contadores y acumuladores

Contador

- **¿Qué hace?** Cuenta cuántas veces ocurre algo
- **Valor inicial:** 0 (o 1)
- **Operación típica:** `contador++` o `contador += 1`
- **Ejemplo:** contar cuántos alumnos aprobaron

Acumulador

- **¿Qué hace?** Suma valores sucesivos
- **Valor inicial:** 0 (valor neutro de la suma)
- **Operación típica:** `total += valor`
- **Ejemplo:** sumar el total de ventas del día

Errores más comunes con acumuladores

→ Olvidar **inicializar en 0** antes del ciclo

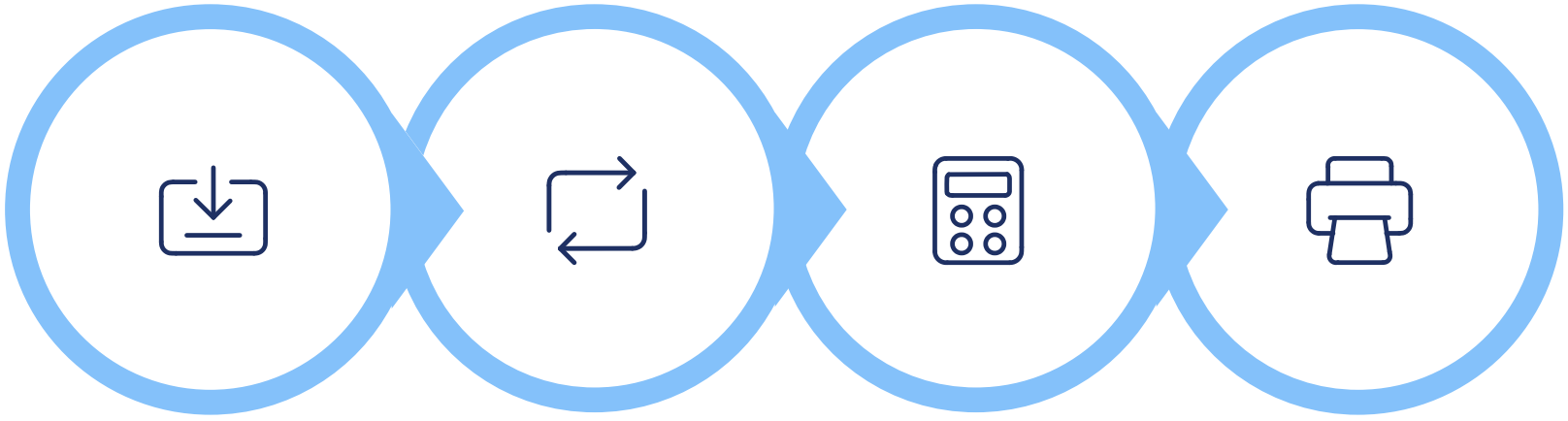
→ Actualizar el acumulador **fuera del ciclo** (solo se suma una vez en lugar de en cada iteración)

→ Mostrar el total **dentro del ciclo** en lugar de después de que este termine

MINI PROYECTO 6

Promedio de temperaturas

Usa un **contador** y un **acumulador** juntos para calcular un promedio. El programa registra las temperaturas de N días, acumula la suma y cuenta cuántos días superaron los 30°C.



Entrada **Ciclo** **Calcular** **Salida**

Traza del acumulador (ejemplo con 4 días)

Día	Temp	Suma (antes)	Suma (después)	Días calurosos
1	28.0	0.0	28.0	0
2	33.0	28.0	61.0	1
3	35.0	61.0	96.0	2
4	22.0	96.0	118.0	2
—	—	promedio = 118.0 / 4	**29.5°C**	—

Hacer esta traza a mano **antes de escribir código** es una de las habilidades más valiosas en programación. Te muestra el valor exacto de cada variable en cada iteración y permite detectar errores antes de ejecutar.

CONCEPTO 5

Entrada y salida: Scanner y printf

`Scanner` es la clase de Java que permite leer datos ingresados por el usuario desde la consola. Siempre debe **importarse** al inicio del archivo y **cerrarse** al final con `sc.close()` para liberar recursos.

`nextInt()`

Lee un número entero. Ideal para edad, cantidad, año.

`nextDouble()`

Lee un número decimal. Ideal para precio, temperatura, nota.

`next()`

Lee una palabra sin espacios. Ideal para respuestas s/n, códigos.

`nextLine()`

Lee una línea completa con espacios. Ideal para nombres y frases.

Si mezclas `nextInt()` o `nextDouble()` con `nextLine()`, puede haber un problema: `nextInt()` deja un salto de línea en el buffer que `nextLine()` lee como línea vacía. **Solución:** agrega un `sc.nextLine()` extra después de `nextInt()` para limpiar el buffer.

printf — mostrar datos con formato

`printf` (*print formatted*) permite controlar exactamente cómo se muestra un número o texto en consola, usando especificadores de formato que se insertan dentro de la cadena de texto.

Especificador	Tipo	Ejemplo de código	Resultado
`%d`	Entero (int)	`printf("%d", 42)`	`42`
`%f`	Decimal (double)	`printf("%f", 3.14)`	`3.141590`
`%.2f`	Decimal 2 decimales	`printf("%.2f", 3.14)`	`3.14`
`%s`	Texto (String)	`printf("%s", "hola")`	`hola`
`%n`	Salto de línea	`printf("fin%n")`	fin + nueva línea
`%5d`	Entero ancho 5	`printf("%5d", 42)`	`42`
`%-20s`	Texto alineado izq	`printf("%-20s", "Ana")`	`Ana`

MINI PROYECTO 7

Ticket de compra

Combina `Scanner` y `printf` para generar una **salida formateada** similar a la de la sumativa. El programa solicita producto, precio unitario y cantidad, calcula subtotal, IVA (19%) y total, y muestra un ticket alineado.

Entrada **Proceso** **Salida** **Cerrar**

El especificador `%-20s` alinea el texto a la izquierda en un campo de 20 caracteres, y `$%10.2f` alinea el importe a la derecha en un campo de 10 caracteres con 2 decimales. Así todas las columnas del ticket quedan perfectamente alineadas.

CONCEPTO 6

Integración: if/else dentro de ciclos

En problemas reales, los ciclos y los `if/else` trabajan juntos. El ciclo **repite un proceso** y el `if/else` toma **decisiones distintas en cada iteración** según el valor actual de los datos. Este es el **patrón central** de la mayoría de los programas de esta unidad.

Inicializar acumuladores y contadores

Antes del ciclo: `acumulador = 0`, `contadorA = 0`, `contadorB = 0`

Ciclo for — iterar N veces

Leer o calcular un valor en cada iteración

if/else — clasificar el valor

Dentro del ciclo: decidir qué categoría corresponde e incrementar el contador correcto

Acumular siempre

La suma del acumulador ocurre en cada iteración, independientemente de la clasificación

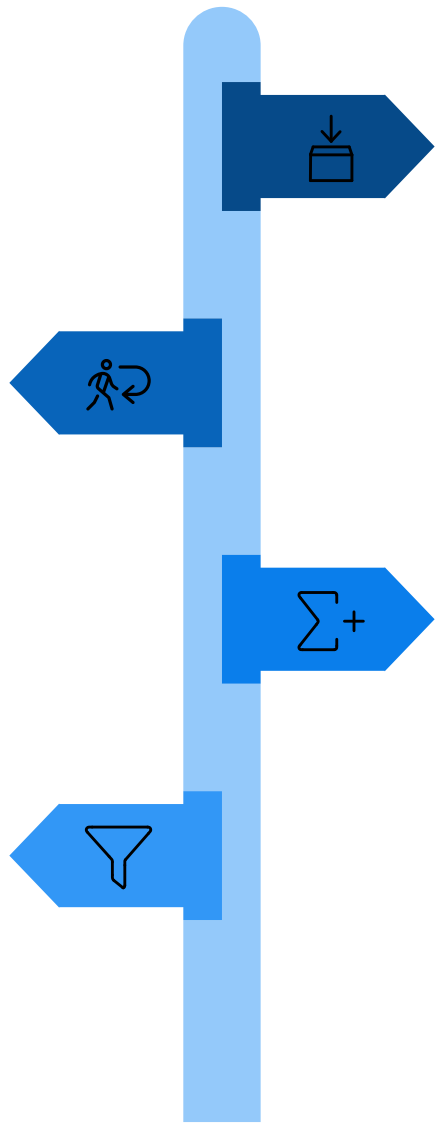
Mostrar resumen — después del ciclo

Imprimir total acumulado, contadores y promedio **fuera** del ciclo

MINI PROYECTO 8

Semáforo de ventas diarias

El proyecto más completo de la guía y el que más se parece al esqueleto de la sumativa. Registra ventas de N días, las clasifica por rango (excelente, regular, bajo) y muestra un resumen con totales y promedios.



Entrada

Leer la cantidad de días N.

Bucle

Para día 1 a N leer ventas del día.

Acumular

totalVentas += ventas del día.

Clasificar

Si >=500000 excelente, >=200000 regular, else bajo y contar.

★ Día excelente

Ventas ⩾ $500.000

◆ Día regular

Ventas ⩾ $200.000

▼ Día bajo

Ventas < $200.000

Este mini proyecto tiene **exactamente el mismo esqueleto** que necesitas para tu sumativa: un `for` que itera N veces, un `if/else` que clasifica dentro del ciclo, un acumulador que crece en cada iteración, y un resumen al salir del ciclo.

Checklist de autoevaluación

Revisa cada punto **antes de subir tu entrega**. Un código que compila pero que tiene errores lógicos puede no aprobar.

Diagramas — E-P-S y Diagrama de Flujo

Diagrama E-P-S completo

- Las tres columnas completas: **Entrada**, **Proceso** y **Salida**
- El Proceso describe tanto la clasificación como el ciclo y el acumulador
- La Salida describe **cada línea** que el programa mostrará
- Hay coherencia entre el E-P-S y el Diagrama de Flujo (misma lógica)

Diagrama de Flujo correcto

- Símbolos correctos: **rombo** para decisión, **rectángulo** para proceso
- Paralelogramo para entrada/salida de datos
- Flechas de vuelta al inicio del ciclo visibles y claras
- Todos los caminos del `if/else` están representados

Código Java — verificación punto a punto

El programa compila y se ejecuta sin errores

Prueba con al menos tres valores diferentes antes de entregar

El if/else cubre todos los rangos sin dejar ninguno sin cubrir

Verifica valores en los bordes exactos de cada rango

El acumulador se inicializa en 0 antes del ciclo y se actualiza dentro

El total final se muestra **después** del ciclo, nunca dentro

Scanner declarado, usado correctamente y cerrado con sc.close()

Los nombres de variables son descriptivos y con convención camelCase

El formato de salida coincide con los ejemplos del enunciado

La indentación es consistente: 4 espacios o 1 tab, nunca mezclados

Pruebas de valores borde — antes de entregar

Estas son las pruebas mínimas que debes ejecutar en tu programa para verificar que los rangos están correctamente implementados.

Temperatura	Rango esperado	Botellas	¿Tu programa lo hace?
−5 °C	⩽ 0 (hace frío)	2	☐ Sí ☐ No
0 °C	⩽ 0 (hace frío)	2	☐ Sí ☐ No
1 °C	1 a 10	Ver tabla	☐ Sí ☐ No
10 °C	1 a 10	Ver tabla	☐ Sí ☐ No
11 °C	11 a 25 (templado)	4	☐ Sí ☐ No
25 °C	11 a 25 (templado)	4	☐ Sí ☐ No
26 °C	26 a 40 (calor)	6	☐ Sí ☐ No
40 °C	26 a 40 (calor)	6	☐ Sí ☐ No
41 °C	> 40 (calor extremo)	8	☐ Sí ☐ No
43 °C	> 40 (calor extremo)	8	☐ Sí ☐ No

Recuerda: esta guía es tu entrenamiento

> Esta guía está diseñada para que construyas el músculo antes de resolver la sumativa. Los mini proyectos usan contextos distintos al enunciado a propósito.

Lo que practicaste

- Variables y tipos de datos
- Estructuras if / else if / else
- Ciclos for y while
- Contadores y acumuladores
- Scanner y printf

Lo que necesitas aplicar

Exactamente esos mismos conceptos, pero con el contexto y los datos del enunciado de tu sumativa. No es nuevo — ya lo conoces.